Static Count-One Ownership for In-Place Array Updates in Lean 4

[Author]

[Date]

Lean statements are quoted from the development; hypotheses are abbreviated where the text says so.

Abstract

Reference counting makes an array update in place exactly when the array's count is one; under any other count the update copies. We ask whether a static discipline can turn that runtime condition into a guarantee for a fragment of Lean 4's compiler, without changing the host language's type system. The development is a shallow embedding: a model heap with counts, a decidable checker whose soundness is proved in Lean, strict recognisers for the emitted array-write prefixes, and a correspondence between the recognised LCNF and the model. We derive the temporary pair's count accounting and the put-back's exclusive branch, build a concrete runtime model in which the exclusivity check cannot abort under the correspondence, and bind the result to the pinned toolchain with an artifact/IR/C gate and a differential corpus. The boundary is explicit: the C runtime's semantics, replay-to-backend identity, the emitted recursive callee and the source correspondence remain assumptions or open obligations.

Argument map

The argument has nine steps. Each is a claim below; the prose introduces why the step is needed, and the Lean statement is the evidence.

Step	Claim	Carrier
1	accepted writes are exclusive	Claim 2
2	the entry invariant supplies count one and separation	Claim 3
3	the emitted write prefix is recognised and corresponds	Claim 4
4	the recognised write lands in place	Claim 5
5	the nested path corresponds without a callee hypothesis	Claim 6
6	the extraction pair returns to count one	Claim 8
7	the put-back's exclusive branch is derived	Claim 9
8	the runtime model cannot abort under the correspondence	Claim 10
9	the linked behaviour agrees on a corpus	Section 8

1 Introduction

Reference-counted functional languages such as Lean 4 and Koka update a shared array by copying it and leave the in-place branch to the runtime's exclusivity test [4, 12, 16]. The resulting cost is data-dependent and invisible in the type: the same source program can copy or not depending on how many references happen to exist at the write. A static discipline that guarantees the count is one at the write would make the in-place branch a property of the program rather than of its current heap, and would let a library or a compiler rule out the copy branch without a dynamic test.

The difficulty is that Lean 4 has no linear, affine or uniqueness types, and adding them would change the kernel, the elaborator and every library. The development therefore does not extend the source type system. It states the count-one property over the compiler's own LCNF, checks it with an ordinary Lean function, and proves that the checked fragment corresponds to a model in which the write is exclusive. The paper makes five contributions. (i) A model heap and a decidable checker whose soundness is a Lean theorem (Claim 2); an entry invariant supplies the heap facts the checker cannot see (Claim 3). (ii) Strict recognisers for the emitted array-write prefixes, with structural rejection of token-preserving near misses. (iii) A correspondence from the recognised LCNF to the model (Claim 4 and Claim 6), including the finite nested path. (iv) A counted extraction/call/restoration argument (Claim 8 and Claim 9). (v) A concrete runtime model and an abort-impossibility theorem (Claim 10), bound to the pinned toolchain by a gate and a differential corpus.

Nothing in the development changes Lean's kernel, elaborator or set of type formers. The discipline is a shallow embedding: `Ctx`, `Heap`, `check`, `run` and their theorems are ordinary Lean definitions; the model programs are `Code`/`NCode` values; the source is Lean, lowered to LCNF, and a recognised prefix is mapped to a model program. A new linear type former would require changing the kernel, unification and every library; the checker avoids that. The price is that linearity is a checked property of a recognised fragment under named contracts rather than a typing judgement for all programs. We state that boundary explicitly.

System	Linearity at the write	Enforced by	Change to the host
Linear Haskell [1]	linear arrows, multiplicities	type checker	language extension
Rust [6]	ownership and borrows	borrow checker	type system
Clean [14]	uniqueness types	type system	language
Alias/ownership types [15, 3, 5]	alias restriction	type system	language
Koka/Perceus [10, 12]	reference counting with reuse	compiler and runtime	compiler
This work	checked count-one discipline	external checker, Lean proofs	none to Lean's core

Table 1: Linearity at the write across disciplines.

Table 1 places the approach. Linear Haskell, Clean, alias types, ownership types and Cyclone change the type system or the language; Rust's borrow checker is part of the type system; Koka and Perceus rely on the compiler's reference counting and reuse. This work leaves the type system alone and checks the count-one condition externally, with proofs in Lean and a correspondence to the emitted code.

2 Setting: Lean 4, LCNF, and reference counting

Lean 4 compiles through a sequence of IRs. The last LCNF phase inserts reference-count operations, and the backend emits C. The runtime decides an array update with `lean_is_exclusive`: count one and non-persistent means in place; every other case copies. The development pins the toolchain, the sibling revision and the captured LCNF artifacts, so every claim names the code it is about. The problem is not to implement reference counting, but to know statically that the count at a particular write is one. Lean's source type system has no linearity or uniqueness; adding one would

be a language change. We therefore state the property over the compiler's code and prove the connection, rather than extend the type system.

2.1 Model programs

The checker does not run on Lean source or on LCNF directly. Its programs are two small inductive types, interpreted over the model heap and environment; the compiler connection is a mapping from the recognised LCNF to these values.

```
inductive LetF where
  | lit (n : Nat)
  | size (a : Var)
  | isShared (x : Var)
  | checkAlias (x : Var)

inductive Code where
  | let (x : Var) (v : LetF) (k : Code)
  | inc (x : Var) (n : Nat) (k : Code)
  | dec (x : Var) (n : Nat) (k : Code)
  | check (x : Var) (k : Code)
  | setArr (x : Var) (i : Var) (v : Var) (k : Code)
  | ret (x : Var)

inductive NCode where
  | base (c : Code)
  | takeOut (a i fst snd : Var) (e : Ref) (k : NCode)
  | putBack (a e i result : Var) (k : NCode)
  | callAt (state path x result : Var) (body : NCode) (k : NCode)
  | ret (x : Var)
```

A `Code` is the straight-line array-write fragment; an `NCode` adds extraction, restoration and the recursive call. Variables are `Nat`, references are `Nat`, `Val` is a reference, a scalar or erased, and a `Heap` maps references to objects with a field array, a count and a persistence flag. `run`/`runN` execute these values and `check`/`checkN` are the checkers; the soundness theorems are about this semantics. They are neither Lean source nor LCNF: `projectPrefix` and `projectNested` walk an LCNF value, `Prefix.toModel` and `indexModelTerm` read a `Code`/`NCode` off the recognised operands, and `execAST`/`execNested` execute the recognised trace. The correspondence theorems relate that execution to the model run.

3 The ownership discipline

3.1 The judgement and its rules

A context records, for each variable, whether it is owned, borrowed, scalar or dead, which reference it denotes, and the multiplicity of that reference. The checker is a total function from a context and a program to a successor context or failure; the frame stack records open extractions. The rules in Table 2 are ownership obligations: a write is admitted only for an owned handle of multiplicity one, an extraction only with the frame discipline, and a call only with an isolated body that returns the state it was handed.

The object language is untyped: there is no type former for a program, and the checker does not infer one by unification. The typing judgement is $c \vdash \mathit{code} \Rightarrow c'$, defined to hold exactly when

`check c code = some c'`; the context is the type of the program point. A context is a usage map, a reference map and a multiplicity map, and the checker is the syntax-directed decision procedure for the rules below.

```
inductive U where
  | owned
  | borrow
  | scalar
  | dead

structure Ctx where
  us    : Var → U
  refOf : Var → Option Ref
  mult  : Ref → Nat

structure Frame where
  arr  : Ref
  idx  : Var
  elem : Ref

def check   : Ctx → Code → Option Ctx
def checkNF : Ctx → List Frame → NCode → Option Ctx
```

The judgement is syntax-directed: each constructor has one rule, the context is an input, and the successor context is computed by `setUs`, `move`, `bindOwned`, `inc` and `dec`. There are no metavariables, no unification and no implicit type arguments in the object language; the only computation is the context update. This is why the discipline makes the count visible at the type level: at the write, the array's handle has usage `owned` and multiplicity one in the context, and the rule refuses every other usage.

The frame stack is the local form of the ownership invariant. An extraction opens a frame naming the array, index and element; a put-back closes the matching frame; a call checks its body under an isolated context and continues with the result. The exit rule is deliberately conservative: with one open frame only the extracted element may return, and a deeper stack must be restored first. These are the rules that make nested updates safe, and they are exactly the parts a source-level linear type system would encode in types; here they are a checker.

The checker sees the shape of the context, not the heap: slot contents, counts, client separation and root completeness are supplied by `Inv` and `EntryInv`, and by the fixture's heap.

3.2 Worked examples

Each example is a `Code` or `NCode` value; the `#guard` is the executable check, and the surrounding context and heap are the repository's fixtures. The first two isolate the write rule; the next two the frame stack; the last two the call's isolation.

Rule	Obligation	Effect
`let x (lit n)`	—	`x` scalar
`let x (size a)`, `let x (isShared a)`	`a` owned or borrow	`x` scalar
`let x (checkAlias a)`	`a` owned, multiplicity one	`x` is the checked alias; `a` dead
`check x`	`x` owned, multiplicity one	unchanged
`setArr x i v`	`x` owned, multiplicity one; `i`, `v` scalar	exclusive write through `x`
`inc x n`	`x` owned	multiplicity $+n$
`dec x n`	`x` owned; $n <$ multiplicity	multiplicity $-n$; `x` dead
`takeOut a i fst snd e`	`a` owned, multiplicity one; `i` scalar; `e` multiplicity one; `fst`, `snd` dead and distinct; `e` ≠ `a`; no open frame at `a`	push the frame; bind parent and element
`putBack a e i result`	`a`, `e` owned, multiplicity one; the frame is `(a,i,e)`; `result` dead	pop the frame; bind the result
`callAt state path x result body`	`state` owned, multiplicity one; `path`, `x` scalar; `result` dead; `CountFree body`; no open frame at `state`; the body is accepted from the isolated context and returns `state`	call the body; bind the result
`ret x`	no frame: `x` owned or scalar; one frame: `x` is the frame's element; deeper: refused	exit

Table 2: The checker's ownership obligations.

Example 1 (A unique write).

```
def execSet (x y i v : Var) : Code :=
  .let y (.checkAlias x) (.setArr y i v (.ret y))

#guard (check (initCtx 0) (execSet 0 3 1 2)).isSome
```

Example 2 (An aliased write is refused).

```
def aliasSet (x y i v : Var) : Code :=
  .inc x 1 (execSet x y i v)

#guard (check (initCtx 0) (aliasSet 0 3 1 2)).isNone
```

The alias increments the handle, so the write's multiplicity-one obligation fails; the same refusal appears at the context level for a heap whose count is two. This is the static counterpart of the runtime copy: the checker rejects the program before it can copy.

Example 3 (An extraction and its return).

```
def good : NCode :=
  .takeOut 0 1 2 3 1 (.ret 3)

#guard (checkN ctx0 good).isSome
```

Example 4 (A stale slot is refused).

```
def bad : NCode :=
  .takeOut 0 1 2 3 1 (.takeOut 2 1 4 5 1 (.ret 5))

#guard (checkN ctx0 bad).isNone
```

The second extraction reopens the array whose slot the first has already emptied. The frame stack records the first extraction, so the second is refused; the runtime would fault at that point, which the fixture also checks.

Example 5 (A recursive body confined to its state).

```
def good : NCode :=
  .takeOut 0 1 2 3 1
    (.callAt 3 4 7 9 (.ret 3) (.ret 9))

#guard (checkN ctx0 good).isSome
```

Example 6 (A suspended parent is refused).

```
def bad : NCode :=
  .takeOut 0 1 2 3 1
    (.callAt 3 4 7 9
      (.takeOut 2 4 5 6 2 (.putBack 5 6 4 8 (.ret 3)))
      (.ret 9))

#guard (checkN ctx0 bad).isNone
```

The refused body returns the call's own state, exactly as the accepted body does; the difference is isolation. The body runs from an isolated context in which the suspended parent is dead, so opening it is not a typing error in the host language but a failure of the checker's ownership obligation.

3.3 The invariant and soundness

The model heap, its counts and exclusivity are the semantic side of the judgement: `Inv` ties the context's multiplicity to the heap count, and `Exclusive` is count one and non-persistence. The checker's soundness theorem is the bridge from the static obligation to the heap: every accepted write selects the exclusive branch. The entry invariant adds the facts a local checker cannot see: the

invariant tree, client separation and root completeness. For fresh arrays `initEntryInv` establishes it, and `checkN_sound_of_entryInv` combines it with the checker.

Claim 1 (Invariant and exclusivity).

```
structure Inv (H : Heap) (env : Env) (c : Ctx) : Prop where
  rc_eq         : ∀ r, H.rc r = c.mult r
  owned         : ∀ x r, c.us x = .owned → c.refOf x = some r →
                    env x = some (.ref r) ∧ 1 ≤ c.mult r
  borrow        : ∀ x r, c.us x = .borrow → c.refOf x = some r →
                    env x = some (.ref r) ∧ H.cells r ≠ none
  scalar        : ∀ x, c.us x = .scalar → ∃ n, env x = some (.scalar n)
  nonPersistent : ∀ r o, H.cells r = some o → o.persistent = false
```

```
def Exclusive (H : Heap) (r : Ref) : Prop :=
  ∃ o, H.cells r = some o ∧ o.rc = 1 ∧ o.persistent = false
```

Claim 2 (Checker soundness).

```
theorem check_sound {c : Code} {ctx ctx' : Ctx} {H : Heap} {env : Env}
    (hc : check ctx c = some ctx') (hi : Inv H env ctx) :
    ∃ H' env' v ws,
      run c env H = .done H' env' v ws ∧
      (∀ w ∈ ws, w.2 = true) ∧ Inv H' env' ctx'
```

Claim 3 (Entry invariant).

```
structure EntryInv (H : Heap) (env : Env) (c : Ctx) (root : Ref)
    (client : List Ref) : Prop extends Inv H env c where
  uniqueTree : UniqueTree H root
  separated  : ClientSeparated H root client
  complete   : RootsComplete c H root
```

```
theorem checkN_sound_of_entryInv
    (hEntry : EntryInv H env c root client)
    (hag : Agrees H env code) (hc : checkN c code = some c') :
    ∃ H' env' v ws,
      runN code env H = .done H' env' v ws ∧
      (∀ w ∈ ws, w.2 = true) ∧ Inv H' env' c'
```

The quoted signature is the repository's; the heap and environment parameters are implicit.

The checker's local rules do not name the heap's path structure. The path theory supplies it: agreement composes through take-out, call and put-back, annotation reachability follows the path, and the safety results lift the checker to every finite path. This is where the per-client proof obligations are replaced by structural facts.

4 Recognising the emitted write

A source-level discipline is only useful if it survives lowering. The compiler rewrites the program, inlines helpers and chooses the emitted shape; the recogniser therefore walks the actual LCNF value, pinning operation order, arities, operand identities and branch constructors. It rejects any token-preserving near miss, so the correspondence is about the code we name, not a look-alike. Three shapes are recognised: the top-level `execP` write, the nested `setAt` path, and the two `Path.left`/`Path.right` pair arms with their joins. The pair arms are the test that the recogniser handles control flow structurally rather than flattening it; the path and the pair carry their own contracts (A1–A12 and N1–N12), named where they are assumed.

5 Correspondence to the ownership model

The checker is about the model, not about the compiler. The bridge executes the operation trace the recogniser emitted for the actual declaration and proves it equal to the model run; the recognition hypothesis is load-bearing because `execAST` returns nothing unless the code decodes to the expected prefix. The primitive meanings are the named contracts, not theorems about Lean's backend.

Claim 4 (Prefix correspondence).

```
theorem project_sound (assign : FVarId → Var) (p : Prefix)
    (code : Code .impure) (hproj : projectPrefix code = some p)
    (hag : Agrees assign p s env H root i v)
    (hcell : H.cells root = some o) (hin : i < o.fields.size) :
    PrefixOutcome.ofExec assign p (execAST assign p code s) =
      some (PrefixOutcome.ofModel (Ownership.run (p.toModel assign) env H))
```

The layout and the heap/environment parameters are implicit, as in the repository.

Claim 5 (The write is in place).

```
theorem recognized_write_in_place
    (hproj : projectPrefix code = some p) (hl : Layout assign p)
    (hEntry : EntryInv H env (Runner.initCtx root) root client)
    (hag : Agrees assign p s env H root i v)
    (hcell : H.cells root = some o) (hin : i < o.fields.size) :
    PrefixOutcome.ofExec assign p (execAST assign p code s) =
        some (inPlaceOutcome root i v o H) ∧
    PrefixOutcome.ofModel (Ownership.run (p.toModel assign) env H) =
        inPlaceOutcome root i v o H
```

The machine, environment and heap are implicit.

The nested path is where a finite model meets recursive code. The model nests take-out, call and put-back per level; the finite-path induction discharges the callee contract from the tail and leaves only the leaf contract and the emitted call's meaning (N10). The model's flat environment cannot express the emitted per-invocation `oproj[1]` index; that bound is recorded rather than hidden.

Claim 6 (Nested finite path).

```
theorem index_path_simulates_model
    (declName takeOut : Name) (assign : FVarId → Ownership.Var)
    (p : NestedPrefix) (code : Code .impure)
    (hproj : projectNested declName takeOut code = some p)
    (ls : List Level) :
    NestedOutcome.ofExec assign p
        (execNestedAST declName takeOut assign body p code s) =
      NestedOutcome.ofModel (Nested.runN (build path value body ls) mEnv H) ∧
    ∃ H' env' ws,
      Nested.runN (build path value body ls) mEnv H =
        .done H' env' (.ref rA) ws ∧ (∀ w ∈ ws, w.2 = true)
```

The level facts, agreement and entry state are implicit; the leaf contract is the remaining supplied premise.

Claim 7 (The finite nest).

```
theorem calls_build (path value : Var) (body : NCode) (ls : List Level) :
    calls (build path value body ls) = calls body + ls.length
```

```
theorem callBody_build (path value : Var) (body : NCode) (l : Level)
    (rest : List Level) :
    callBody? (build path value body (l :: rest)) =
      some (build path value body rest)
```

6 Counted extraction, call, and restoration

The emitted take-out creates a pair whose fields are the parent and the extracted child. The constructor moves the handles into the pair; the projections increment them; freeing the unique pair releases both edges. The counts are therefore equal at the boundary, but only because the pair is freed: the negative controls show that omitting the free, or retaining an alias, leaves a count of two. Without this accounting, N2's neutrality would be a modeling premise rather than a derived fact.

Claim 8 (Pair accounting).

```
theorem pair_accounting {H : Heap} {p c : Ref} {i : Nat}
    (hw : WellFormedEntry H p c i) :
    let B := protocol H p c i
    B.2.rc p = 1 ∧ B.2.rc c = 1 ∧
      B.2.cells p ≠ none ∧ B.2.cells c ≠ none ∧
      B.2.cells B.1 = none ∧
      Heap.Exclusive B.2 p ∧ Heap.Exclusive B.2 c ∧
      B.2.rc p = H.rc p ∧ B.2.rc c = H.rc c
```

```
theorem no_free_components_not_exclusive
    (hw : WellFormedEntry H p c i) :
    let B := protocolNoFree H p c i
    B.2.rc p = 2 ∧ B.2.rc c = 2 ∧ ¬ Heap.Exclusive B.2 p
```

The put-back is the second place where a copy can hide. The operation has two dispositions: at count one it mutates the parent; at a higher count or a persistent parent it allocates a fresh cell and decrements the input. The exclusive branch is derived from the counted-pair boundary and the child-call boundary, with functional restoration and count preservation separate; the returned-handle accounting remains a named premise.

Claim 9 (Put-back exclusivity).

```
theorem putBack_exclusive_of_wellFormed
    (hw : CountedPair.WellFormedEntry H p c i)
    (hc : ChildCall Hin Hout env ctx state path x body p c) :
    ∃ H',
      putBack H p i c = some (p, H') ∧
      H'.cells p = (mutate H p o i c).cells p ∧
      Heap.Exclusive H' p ∧ H.rc p = 1
```

The child's admitted body and the returned-handle accounting are implicit; the conclusion is the exclusive branch and the parent's count.

7 Runtime model and abort impossibility

The model's count is a ghost count; the runtime's is the one that decides. The runtime model is a separate state machine with its own heap, counts, persistence, guarded allocator, check and two array-set dispositions. The structural correspondence preserves refs, cells, counts, persistence, fields and the cursor; from it the model's exclusivity implies the runtime's, the check returns, and the write mutates. The abort branch is unreachable under the correspondence and `EntryInv`*.*

The definitions are ordinary Lean structures. The correspondence contains no exclusivity or disposition; those are derived from the preserved counts, persistence and fields. A guarded allocator refuses a dirty cursor, so the copy branch cannot overwrite a live cell.

Claim 10 (No abort).

```
theorem checkExclusive_no_abort {H : Heap} {R : RHeap} {r : Ref}
    (hC : Corr H R) (hx : H.Exclusive r) :
    checkExclusive R r = .ok r ∧
      ∀ r', checkExclusive R r ≠ .abort r'
```

```
theorem setAt_inPlace_of_entryInv
    (hEntry : EntryInv H env (Runner.initCtx root) root client)
    (hC : Corr H R) (hcell : H.cells root = some o)
    (hR : R.cells root = some o') (hin : i < o.fields.size) :
    ∃ H',
      setAt R root i (.scalar vn) = some (.inPlace H' root) ∧
      Corr (inPlaceOutcome root i vn o H).heap H'
```

Claim 11 (Emitted-semantics abort impossibility).

```
def PrefixAborts (out : Option PrefixOutcome) : Prop :=
  ∃ o, out = some o ∧ o.fault ≠ none

theorem prefix_abort_impossible
    (hproj : projectPrefix code = some p)
    (hEntry : EntryInv H env (Runner.initCtx root) root client)
    (hag : Agrees assign p s env H root i v)
    (hcell : H.cells root = some o) (hin : i < o.fields.size) :
    ¬ PrefixAborts (PrefixOutcome.ofExec assign p (execAST assign p code s))
```

The layout and the remaining entry premises are implicit; this is the emitted-semantics form of the no-abort result.

8 Empirical alignment

The model is only as good as its match to the compiler. A compiled probe compares the model's predicates with `lean_is_exclusive` *and address identity on a corpus: a unique array, a live alias, the alias released before the write, a persistent literal and a closure-held array. The closed-pure-array case is the important negative: a pure closed array is lifted to a* `___closed` *constant read through* `lean_obj_once`*, so it is shared and the write copies even though the source looks unique. The gate also binds the pinned artifacts, the IR shapes, the generated C and the runtime header. The corpus is evidence, not an exhaustive equivalence.*

Case	*Predicted*	*Observed*
unique array	*exclusive, in place*	*agree*
live alias (count two)	*shared, copy*	*agree*
alias released before the write	*exclusive, in place*	*agree*
persistent literal	*shared, copy*	*agree*
closure-held array	*shared, copy*	*agree*

9 Limits and non-claims

Non-claim 1.
The boundary is explicit. The C semantics of the primitives is not formalised; replay-to-backend identity (A10) and address identity remain assumptions; the emitted recursive callee and continuation (N10) are not captured; the model's flat environment cannot express the emitted per-invocation index; the leaf body contract and several heap-shape facts remain supplied; free/cascade, persistence, concurrency and callbacks are outside the fragment; and the M7 source correspondence is not attempted.

10 Related work

Reference counting and reuse: Perceus and Counting Immutable Beans [12, 16], and Koka's effect typing [10]. Linear and uniqueness disciplines: Wadler's linear types [17], Linear Haskell [1], Clean's uniqueness [14], alias types [15], ownership types [3] and Cyclone [5]. Program logics: separation logic [13], Iris [7], RustBelt [6] and Verus [9]. Verified compilation and kernels: CompCert [11], CakeML [8], Lean 4 [4] and Lean4Lean [2]. The contrast is the one drawn in Table 1: these systems put linearity in the type system, the borrow checker or the compiler, while this development checks the emitted fragment and proves the connection in Lean.

11 Conclusion

The development proves the count-one discipline at the model, the recognised LCNF and the runtime model, under named contracts; it measures the linked primitive on a corpus and gates the pinned evidence. What remains is the C semantics, the replay/backend identity, the emitted callee and the source correspondence. The next step is either a verified runtime model or a stronger backend gate; the static discipline itself is complete for the recognised fragment.

A Theorem map and artifacts

The table lists the headline declarations and their status. "Proved" means a kernel-checked theorem; "contracts" and "leaf/N10" name the assumptions under which the theorem is stated; "measured" means the compilation gate and the corpus, not a proof.

`check_sound`
`Ownership/Soundness.lean` — *proved.*

`checkN_sound_of_entryInv`
`Ownership/Entry.lean` — *proved.*

`project_sound`
`OwnershipLcnf/Semantics.lean` — *proved under the primitive contracts.*

`recognized_write_in_place`
`OwnershipLcnf/Semantics.lean` — *proved under the primitive contracts.*

`index_path_simulates_model`
`OwnershipLcnf/PathRecursion.lean` — *proved; leaf contract and N10 remain.*

`pair_accounting`
`Ownership/CountedPair.lean` — *proved; restricted free.*

`putBack_exclusive_of_wellFormed`
`Ownership/PutBack.lean` — *proved; returned-handle accounting remains.*

`checkExclusive_no_abort`
`OwnershipLcnf/RuntimeModel.lean` — *proved in the runtime model.*

`prefix_abort_impossible`
`OwnershipLcnf/NoAbort.lean` — *proved under A6/A8.*

`differential corpus`
`runtime-bridge` — *measured.*

References

[1] Jean-Philippe Bernardy, Mathieu Boespflug, Ryan R. Newton, Simon Peyton Jones, and Arnaud Spiwack. Linear haskell: Practical linearity in a higher-order polymorphic language. In Principles of Programming Languages (POPL)*, 2018.*

[2] Mario Carneiro. Lean4lean: Towards a formalized proof checker for lean 4, 2024. Preprint.

[3] Dave Clarke, John Potter, and James Noble. Ownership types for flexible alias protection. In Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, 1998.*

[4] Leonardo de Moura and Sebastian Ullrich. The lean 4 theorem prover and programming language. In Automated Deduction – CADE-28*, volume 12699 of* LNCS*. Springer, 2021.*

[5] Trevor Jim, J. Gregory Morrisett, Dan Grossman, Michael W. Hicks, James Cheney, and Yanling Wang. Cyclone: A safe dialect of c. In USENIX Annual Technical Conference*, 2002.*

[6] *Ralf Jung, Jacques-Henri Jourdan, Robbert Krebbers, and Derek Dreyer. Rustbelt: Securing the foundations of the rust programming language. In* Principles of Programming Languages (POPL)*, 2018.*

[7] *Ralf Jung, David Swasey, Filip Sieczkowski, Kasper Svendsen, Aaron Turon, and Lars Birkedal. Iris: Monoids and invariants as an orthogonal basis for concurrent reasoning. In* Principles of Programming Languages (POPL)*, 2015.*

[8] *Ramana Kumar, Magnus O. Myreen, Michael Norrish, and Scott Owens. Cakeml: A verified implementation of ml. In* Principles of Programming Languages (POPL)*, 2014.*

[9] *Andrea Lattuada, Travis Hance, Chanhee Cho, Matthias Brun, Isitha Subasinghe, Yi Zhou, Jon Howell, Bryan Parno, and Chris Hawblitzel. Verus: Verifying rust programs using linear ghost types. In* Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, 2023. Verify author list.*

[10] *Daan Leijen. Type directed compilation of row-typed algebraic effects. In* Principles of Programming Languages (POPL)*, 2017.*

[11] *Xavier Leroy. Formal verification of a realistic compiler.* Communications of the ACM*, 52(7), 2009.*

[12] *Alex Reinking, Ningning Xie, Leonardo de Moura, and Daan Leijen. Perceus: Garbage free reference counting with reuse. In* Programming Language Design and Implementation (PLDI)*, 2021.*

[13] *John C. Reynolds. Separation logic: A logic for shared mutable data structures. In* Logic in Computer Science (LICS)*, 2002.*

[14] *Sjaak Smetsers, Erik Barendsen, Marko van Eekelen, and Rinus Plasmeijer. Uniqueness typing for functional languages with graph rewriting semantics.* Mathematical Structures in Computer Science*, 1996. Verify volume and pages.*

[15] *Frederick Smith, David Walker, and J. Gregory Morrisett. Alias types. In* European Symposium on Programming (ESOP)*, 2000.*

[16] *Sebastian Ullrich and Leonardo de Moura. Counting immutable beans: Reference counting optimized for purely functional programming. In* Implementation and Application of Functional Languages (IFL)*, 2019.*

[17] *Philip Wadler. Linear types can change the world!* Programming Concepts and Methods*, 1990.*